

Compte Rendu TP 4 : Problème du Sac à Dos

L3 Informatique - Programmation Parallèle (OpenMP)

1. Implémentation Séquentielle

Pour résoudre le problème du sac à dos, on utilise la programmation dynamique. L'idée est de construire une matrice $S[\text{objets}][\text{capacité}]$.

Points importants :

- **Mémoire :** Comme les fichiers de test (surtout pb6) sont gros, j'ai alloué la matrice dynamiquement avec `malloc`. Sinon, on risquait un "Stack Overflow".
- **Vérification :** À la fin du programme, une fonction vérifie que la solution reconstruite (tableau E) correspond bien à l'utilité maximale trouvée dans la matrice.
- **Valgrind :** J'ai passé le code au Memcheck, il n'y a aucune fuite mémoire.

2. Passage en Parallèle avec OpenMP

Dans l'algorithme, chaque ligne dépend de la ligne précédente. On ne peut donc pas paralléliser la boucle extérieure (celle des objets). Par contre, on peut paralléliser le remplissage des colonnes (les capacités) car elles sont indépendantes sur une même ligne.

J'ai optimisé la parallélisation en mettant le `#pragma omp parallel` autour de toutes les boucles plutôt que de le recréer à chaque objet. Ça permet de gagner du temps en évitant de "forker" les threads 96 fois.

```
#pragma omp parallel { // Remplissage de la ligne 0
#pragma omp for
for (int j = 0; j <= p->capacity; j++) { ... } // Remplissage du
reste
for (int i = 1; i < p->numObj; i++) { #pragma omp for
for (int j = 0; j <= p->capacity; j++) { // Calcul de S[i][j] } } }
```

3. Résultats et Mesures

Voici les résultats obtenus sur le fichier `pb6.txt` (96 objets, $C=640\,400$). Les temps sont une moyenne sur 5 lancements.

Threads	Temps Moyen (s)	Speedup
Séquentiel	0.119s	1.00x
2 Threads	0.084s	1.41x
4 Threads	0.047s	2.52x
8 Threads	0.063s	1.89x
16 Threads	0.057s	2.10x

Observation : Le meilleur résultat est obtenu avec 4 threads. Au-delà, le gain n'est pas linéaire et les performances stagnent.

4. Analyse et Conclusion

Pourquoi le speedup n'est pas de 16x avec 16 threads ?

Le programme est "Memory Bound". Ça veut dire que le CPU passe plus de temps à attendre que les données arrivent de la RAM qu'à faire les calculs (qui sont juste une addition et un max). À partir d'un certain nombre de threads, le bus mémoire est saturé et ajouter des coeurs n'aide plus.

Petits fichiers (pb1, pb2) :

Pour ces fichiers, la version parallèle est plus lente que la séquentielle. C'est normal : le temps de créer les threads est plus long que le calcul lui-même. C'est ce qu'on appelle l'overhead d'OpenMP.

Conclusion :

Le TP est fonctionnel et la parallélisation fonctionne bien sur les grosses instances. L'optimisation du pool de threads a permis d'améliorer le speedup global.